yana-bhuyan / ASSIGNMENTS

Code Issues Pull requests Actions Projects Wiki Security Insig

main ASSIGNMENTS / dsa_lab_2 / lab.md

yana-bhuyan Add files via upload 14b7144 · now

77 lines (69 loc) · 3.03 KB

Preview Code Blame Raw

Student name : Swayam Uddeepta Bhuyan Sap id: 590021784 Batch : B-17 Course : Btech Cse Subject : DSA Experiment : 2

Experiment 2:Linked List Data Structure and its Applications

Objective: To experiment the concept of pointers, structure and dynamic memory allocation to realize linked list and its application.

1) Implement single Linked List data structure and its operations like insert and delete in the beginning/end and nth position of the list, and display the items stored in the linked list.

Aim:

To implement a Single Linked List and perform operations such as: Insertion at beginning Insertion at end Insertion at nth position Deletion at beginning Deletion at end Deletion at nth position Display the elements of the linked list

Algorithm:

1.Start 2.Initialize head pointer to NULL 3.Create a node with given data 4.For insertion at beginning, link new node before head and update head 5.For insertion at end, traverse list to last node and attach new node 6.For insertion at nth position, traverse to (n-1) node and insert new node 7.For deletion at beginning, move head to next node and delete first node 8.For deletion at end, traverse to second last node and delete last node 9.For deletion at nth position, traverse to (n-1) node and delete nth node 10.Traverse the linked list from head to NULL and display elements 11.Stop

Program:

C ⌗ > main()

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

// Insert at beginning
void insertBegin(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}

// Insert at end
void insertEnd(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }
```

```
    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

// Insert at position
void insertPos(int val, int pos) {
    if (pos == 1) {
        insertBegin(val);
        return;
    }

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;

    struct Node* temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL) return;

    newNode->next = temp->next;
    temp->next = newNode;
}
```

```
}

// Delete beginning
void deleteBegin() {
    if (head == NULL) return;

    struct Node* temp = head;
    head = head->next;
    free(temp);
}

// Delete end
void deleteEnd() {
    if (head == NULL) return;

    if (head->next == NULL) {
        free(head);
        head = NULL;
        return;
    }

    struct Node* temp = head;
    while (temp->next->next != NULL)
        temp = temp->next;

    free(temp->next);
    temp->next = NULL;
```

```c
    temp->next = NULL;
}

// Delete at position
void deletePos(int pos) {
    if (pos == 1) {
        deleteBegin();
        return;
    }

    struct Node* temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL || temp->next == NULL) return;

    struct Node* del = temp->next;
    temp->next = del->next;
    free(del);
}

// Display list
void display() {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
```

```c
void display() {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {

    insertBegin(10);
    insertBegin(5);
    insertEnd(20);
    insertPos(15, 3);

    display();

    deleteBegin();
    deleteEnd();
    deletePos(2);

    display();

    return 0;
}
```

Compilation and Execution:

```
HP@DESKTOP-7K7BG1T MINGW64 ~/OneDrive - UPES/Desktop/dsa_c/dsa_lab_2
$ ./1
HP@DESKTOP-7K7BG1T MINGW64 ~/OneDrive - UPES/Desktop/dsa_c/dsa_lab_2
$ ./1
$ ./1
5 -> 10 -> 15 -> 20 -> NULL
10 -> NULL

10 -> NULL
```

Observation:

Linked list uses dynamic memory allocation.

Insertion and deletion are easier compared to arrays.

Pointer manipulation is essential.

Traversal is required to access elements.

Result:

The single linked list was successfully implemented and operations like insertion, deletion, and display were performed correctly.

2)Using single linked list and functions implement Stack and its operations like insert, delete, and display.

Aim:

To implement Stack using Single Linked List and perform operations such as push, pop, and display.

Algorithm:

1.Start 2.Initialize top pointer to NULL 3.push operation, create a new node and insert it at beginning (update top) 4.For pop operation, delete the first node and update top 5.For display, traverse from top to NULL and print elements 6.Stop

Program:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
```

OneDrive - UPES\Desktop\dsa_c\dsa_lab_2\2

```c
    struct Node* next;
};

struct Node* top = NULL;

// PUSH
void push(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = top;
    top = newNode;
}

// POP
void pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }

    struct Node* temp = top;
    top = top->next;
    free(temp);
}
```

```c
}

// DISPLAY
void display() {
    if (top == NULL) {
        printf("Stack is empty\n");
        return;
    }

    struct Node* temp = top;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {

    push(10);
    push(20);
    push(30);

    display();

    pop();
    display();

    return 0;
}
```

Compilation and Execution:

```
HP@DESKTOP-7K7BG1T MINGW64 ~/OneDrive - UPES/Desktop/dsa_c/dsa_lab_2
$ gcc -o 2 2.c

HP@DESKTOP-7K7BG1T MINGW64 ~/OneDrive - UPES/Desktop/dsa_c/dsa_lab_2
$ ./2
30 -> 20 -> 10 -> NULL
20 -> 10 -> NULL
```

Observation:

Stack follows LIFO principle.

Push and pop happen at the beginning.

Linked list allows dynamic memory allocation.

Result:

Stack was successfully implemented using a linked list and push, pop, and display operations were performed.